

Process Management Utilities

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

Dr. Joseph Issa

November, 24th, 2024

Process Management Utilities

Process management is the topic of the week. Both Linux and Windows provide tools to allow a user to do this manually, and the operating systems (OS) differ significantly in terms of design theory. With its command line terminal interface, Linux offers unmatched flexibility and control. Windows, on the other hand, employs a user-friendly graphical user interface (GUI) task manager and the more advanced process explorer. Today, I will explore and compare the two paradigms using Fedora to demonstrate the capabilities of a Linux OS alongside Windows 11. While most users are familiar with basic process management tasks such as listing running processes or terminating them, there is a lot more that can be accomplished to showcase the unique features along with the strengths and weaknesses of each OS' processes management systems.

Process Isolation

Process isolation (PI) demonstrates how an OS segregates and manages processes to allocate resource ownership, prevent interference, and ensure stability. Linux and Windows have distinct ways of achieving isolation. In Linux, PI is achieved using Process ID (PID) Namespaces. Processes inside a namespace have their own PID hierarchy. Within the *util-linux* package, the *unshare* command can be used to isolate a PID namespace. It turns out that *util-linux* comes installed by default on Fedora, and I did not have to go through the trouble of using *dnf install util-linux*, but it's interesting to note that RedHat doesn't use *apt-get* that I am used to using in Debian based distributions. We are changing system configurations here, so we'll have to use the superuser modifier *sudo* to run the command with administrative privileges.

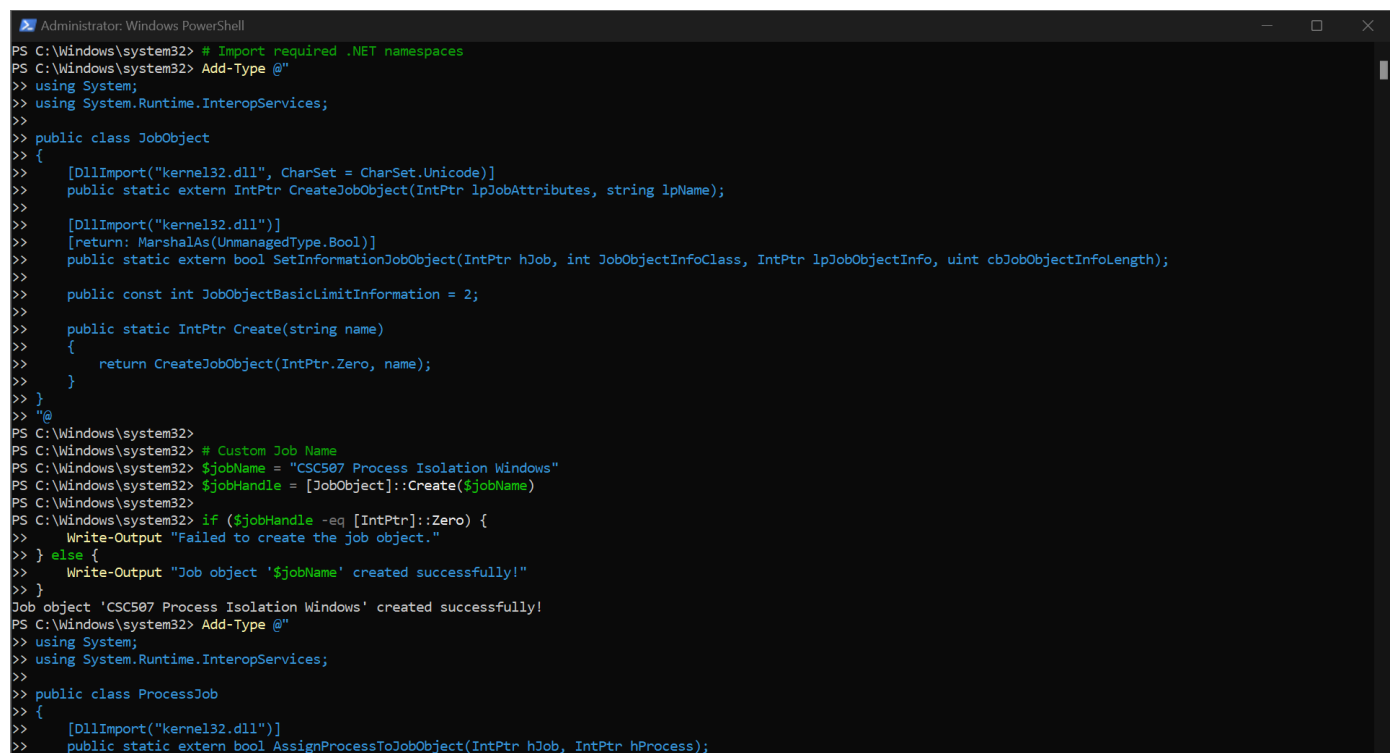
The command **`sudo unshare --pid --fork bash`** will allow Fedora to execute a program in a new namespace, **`--pid`** specifies to create a new PID namespace and gives processes an independent PID hierarchy starting with PID 1, **`--fork`** starts a child process, and **`bash`** becomes PID 1 within the new namespace (Ahao, 2024.) After this is created, running the command **`ps`** shows we have a new PID 1 **`systemd`**.

```
jwensink@vbox:~$ sudo unshare --pid --fork bash
[sudo] password for jwensink:
root@vbox:/home/jwensink# ps
  PID TTY          TIME CMD
    1 ?           00:00:03 systemd
```

I was actually expecting to see **`bash`** here instead of **`systemd`**, it turns out Fedora automatically invokes **`systemd`** when **`sudo`** spawning a new namespace when using the **`unshare`** command as the default **`init`** system. This is tied to Fedora's use of **`systemd`** to handle other process management tasks like reaping orphaned child processes more effectively than a basic bash shell.

```
jwensink@vbox:~$ ps aux | grep bash
jwensink    5323  0.0  0.0 233048  5668 pts/0    Ss   11:08   0:00 /usr/bin/bash
root        5399  0.0  0.1 244460  9268 pts/0    S+   11:09   0:00 sudo unshare --pid --fork bash
root        5448  0.0  0.0 244460  2924 pts/1    Ss   11:09   0:00 sudo unshare --pid --fork bash
root        5449  0.0  0.0 229480  1708 pts/1    S    11:09   0:00 unshare --pid --fork bash
root        5450  0.0  0.0 232912  5820 pts/1    S+   11:09   0:00 bash
jwensink    5531  0.0  0.0 233048  5856 pts/2    Ss+  11:14   0:00 /usr/bin/bash
jwensink    6123  0.0  0.0 233048  5796 pts/3    Ss   11:27   0:00 /usr/bin/bash
jwensink    6218  0.0  0.0 230336  2232 pts/3    S+   11:28   0:00 grep --color=auto bash
jwensink@vbox:~$
```

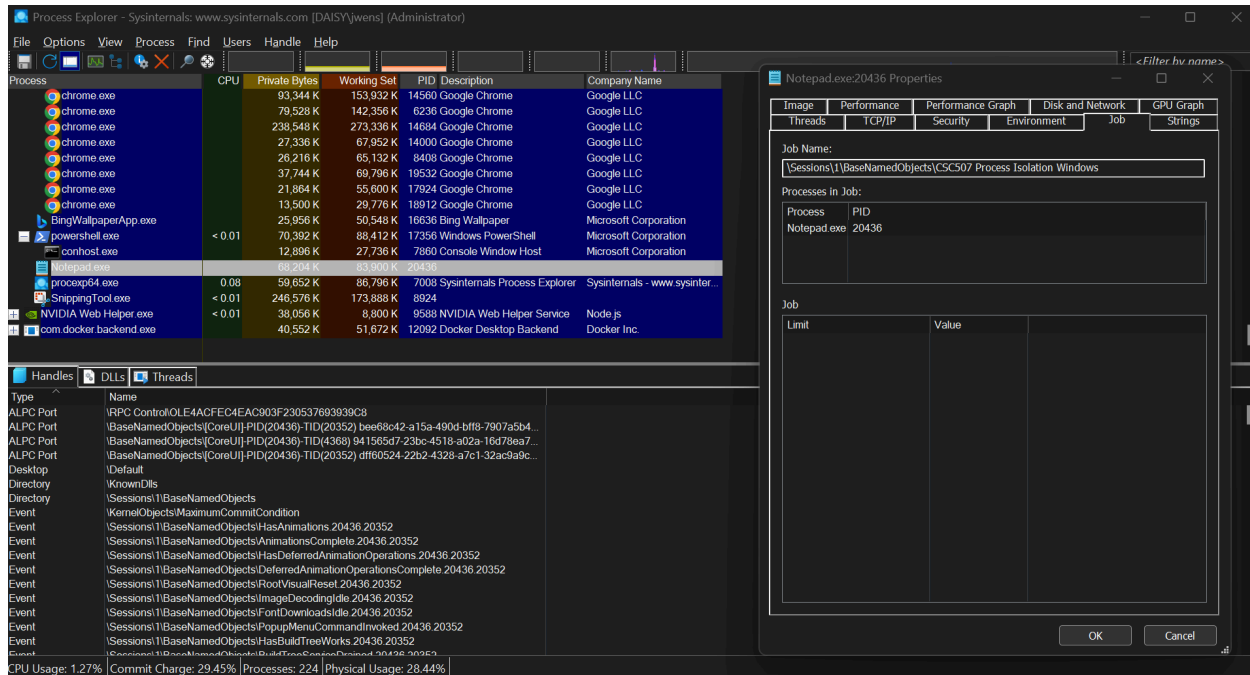
In Windows, process isolation works a bit differently. Windows doesn't have a direct equivalent to Linux namespaces, but there are a few workarounds (Microsoft, 2023.) Windows can create a job object and set restrictions on memory usage, similar to how Linux namespaces can isolate a process. Using Powershell, we can import the .NET framework and Windows APIs to create a new job object using the **CreateJobObject** function. Once the job object was created, we used the **AssignProcessToJobObject** function to associate an already running process (Notepad, for this example) with the job object. I used ChatGPT (OpenAI, 2024) to generate the script for me, and it did an excellent job.



```
Administrator: Windows PowerShell
PS C:\Windows\system32> # Import required .NET namespaces
PS C:\Windows\system32> Add-Type @"
>> using System;
>> using System.Runtime.InteropServices;
>>
>> public class JobObject
>> {
>>     [DllImport("kernel32.dll", CharSet = CharSet.Unicode)]
>>     public static extern IntPtr CreateJobObject(IntPtr lpJobAttributes, string lpName);
>>
>>     [DllImport("kernel32.dll")]
>>     [return: MarshalAs(UnmanagedType.Bool)]
>>     public static extern bool SetInformationJobObject(IntPtr hJob, int JobObjectInfoClass, IntPtr lpJobObjectInfo, uint cbJobObjectInfoLength);
>>
>>     public const int JobObjectBasicLimitInformation = 2;
>>
>>     public static IntPtr Create(string name)
>>     {
>>         return CreateJobObject(IntPtr.Zero, name);
>>     }
>> }
>> @"
PS C:\Windows\system32>
PS C:\Windows\system32> # Custom Job Name
PS C:\Windows\system32> $jobName = "CSC507 Process Isolation Windows"
PS C:\Windows\system32> $jobHandle = [JobObject]::Create($jobName)
PS C:\Windows\system32>
PS C:\Windows\system32> if ($jobHandle -eq [IntPtr]::Zero) {
>>     Write-Output "Failed to create the job object."
>> } else {
>>     Write-Output "Job object '$jobName' created successfully!"
>> }
Job object 'CSC507 Process Isolation Windows' created successfully!
PS C:\Windows\system32> Add-Type @"
>> using System;
>> using System.Runtime.InteropServices;
>>
>> public class ProcessJob
>> {
>>     [DllImport("kernel32.dll")]
>>     public static extern bool AssignProcessToJobObject(IntPtr hJob, IntPtr hProcess);
```

Task Manager is Windows' default process monitoring system and is fine for most purposes, but a less commonly used system, Process Explorer provides more advanced capabilities beyond what Task Manager can offer, in this case verifying the association of the notepad process with our newly created job object. Process Explorer confirmed that the notepad.exe process was successfully linked to the

“CSC507 Process Isolation Windows” job object. If we wanted to restrict CPU or memory usage or select a processor core affinity for the job object, Task Manager would offer some limited functionality to implement restrictions. However, for implementing more precise restrictions, another PowerShell script would have needed to be created.



Monitor Disk I/O Processes

There are many different approaches to monitoring disk I/O processes in Linux (Unix and Linux Stack Exchange, 2012.) One could use a built-in command like *vmstat* to provide a quick overview of disk performance, including an overview of blocks read and written per second.

Alternatively, the command

cat /proc/diskstats would allow a user to view raw data about disk activity directly from the

kernel. For information about a specific process, the command *cat /proc/<PID>/io* if you know

the PID of the process you wish to monitor. There are also numerous tools like *iostat*, or *sar* that

are available as part of the *sysstat* package, which I would recommend to anybody taking a Linux OS class. These tools offer more detailed functionality, enabling a user to view I/O statistics over time or for specific devices. Additionally, *dstat* provides real-time and customizable views of I/O activity metrics. Another useful tool is the *iostat* package, which provides a view similar to the *top* command for monitoring CPU and memory usage. For a more visual representation, tools like *atop* and *htop* include I/O metrics alongside processor and memory statistics for a more in-depth monitoring experience.

```

CSC507-Fedora [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Nov 24

jwensink@vbox:~$
Proceed with changes? [N/y] y

* Waiting in queue...
* Waiting for authentication...
* Waiting in queue...
* Downloading packages...
* Requesting data...
* Testing changes...
* Installing packages...
Linux 6.11.4-301.fc41.x86_64 (vbox) 11/24/2024 _x86_64_ (8 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           1.48    0.01    1.88    0.31    0.00   96.32

Device            tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
sda                41.00     4397.85     219.53         0.00    2422729    153992      0
sr0                 0.17         5.72         0.00         0.00       3150      0         0
zram0              0.10         2.14         0.01         0.00       1180      4         0

jwensink@vbox:~$ vmstat
procs-----memory----- --swap-- -----io----- -system-- -----cpu-----
 r b swpd free buff cache si so bi bo in cs us sy id wa st gu
 9 0  0 2991876 4680 3412228 0 0 4130 288 1922 2 1 2 96 0 0 0
jwensink@vbox:~$ iostat
Linux 6.11.4-301.fc41.x86_64 (vbox) 11/24/2024 _x86_64_ (8 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           1.42    0.01    1.83    0.30    0.00   96.44

Device            tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
sda                40.37     4077.79     284.87         0.00    2433897    169972      0
sr0                 0.16         5.28         0.00         0.00       3150      0         0
zram0              0.09         1.98         0.01         0.00       1180      4         0

jwensink@vbox:~$ cat /proc/diskstats
 8      0 sda 16486 8434 4866194 35422 7878 5691 347248 22937 0 16213 58989 0 0 0 0 862 630
 8      1 sda1 26 0 208 33 0 0 0 0 32 33 0 0 0 0 0
 8      2 sda2 262 35 13826 363 48 21 464 125 0 312 488 0 0 0 0 0
 8      3 sda3 16036 8399 4845432 34761 7828 5670 346784 22804 0 18680 57566 0 0 0 0 0
11     96 sr0 6300 169 0 0 0 0 0 140 169 0 0 0 0 0
252    0 zram0 54 0 2360 0 1 0 8 0 0 0 0 0 0 0 0

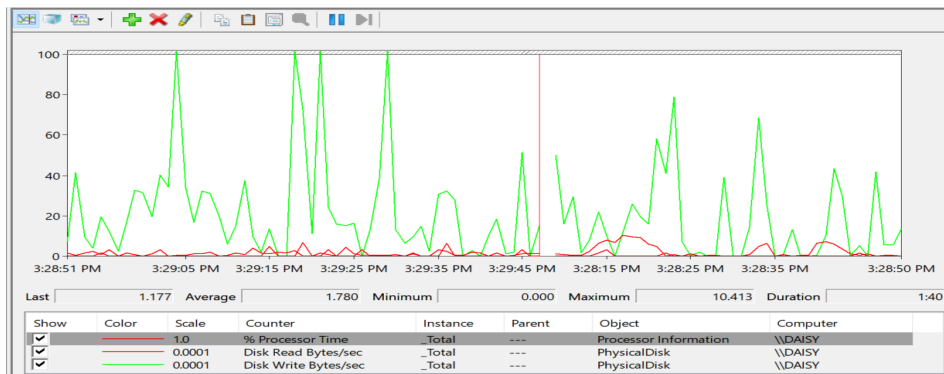
```

For Windows, the Task Manager does a decent job of monitoring I/O operations for individual processes. By enabling columns for I/O Reads and Writes, I/O Bytes Read and

Written in the details tab, a user can get a great graphical representation of how much data each process is reading or writing to the disk. What’s nice about this is you can sort by name, PID, or utilization, depending on your use case.

PID	Status	User name	CPU	Memory (ac...	I/O reads	I/O writes	I/O re...	I/O wr...
3936	Running	SYSTEM	00	756 K	2	2	894.8...	52,964
3996	Running	hwens	00	17,516 K	39,076	30,378	4,881,...	3,533,...
4116	Running	SYSTEM	00	21,608 K	6,289	1,627	224.3...	4,588,...
4128	Running	NETWORK ...	00	2,580 K	1,528	61	1,122,...	18,426
4144	Running	SYSTEM	00	8,500 K	192	84	20.52...	22,87...
4160	Running	SYSTEM	00	1,916 K	0	0	0	0
4192	Running	SYSTEM	00	496 K	8	0	128	0
4196	Running	SYSTEM	00	1,500 K	6	2	115.1...	72
4204	Running	SYSTEM	00	228 K	45	0	23,040	0
4344	Running	SYSTEM	00	724 K	0	0	0	0
4392	Running	SYSTEM	00	126,528 K	24,465	14,939	4,314,...	546.8...
4456	Running	SYSTEM	00	4,372 K	424	9	3,326,...	19,844
4464	Running	SYSTEM	00	1,940 K	5	0	1,214	0
4480	Running	LOCAL SER...	00	1,912 K	0	0	0	0
4488	Running	LOCAL SER...	00	884 K	0	0	0	0
4612	Running	hwens	00	38,700 K	17,909	25,331	3,026,...	6,075,...
4624	Running	SYSTEM	00	4,968 K	1,951	3	931.8...	2,836
4648	Running	SYSTEM	00	1,616 K	0	0	0	0
4696	Running	SYSTEM	00	5,476 K	82	2	34,712	320
4720	Running	LOCAL SER...	00	11,496 K	1	2	116	32,928
4764	Running	NETWORK ...	00	896 K	261	192	15,632	19,184
5096	Running	SYSTEM	00	576 K	0	0	0	0
5112	Running	SYSTEM	00	740 K	1	2	0	806
5356	Running	SYSTEM	00	944 K	204	137	14,208	32,256
5740	Running	hwens	00	700 K	618	616	90,085	90,085
6236	Running	hwens	00	46,920 K	7,379	8,114	73,60...	3,562,...
6448	Running	SYSTEM	00	552 K	0	0	0	0

If more advanced metrics are needed, the Performance Monitor is another built in tool that provides a tremendous amount of data about I/O operations. Performance Monitor allows users to customize counters which makes it ideal for longer-term monitoring or detailed analysis of disk activity. Performance monitor has a graphical view and a textual view that can be customized to provide advanced insight into system performance.



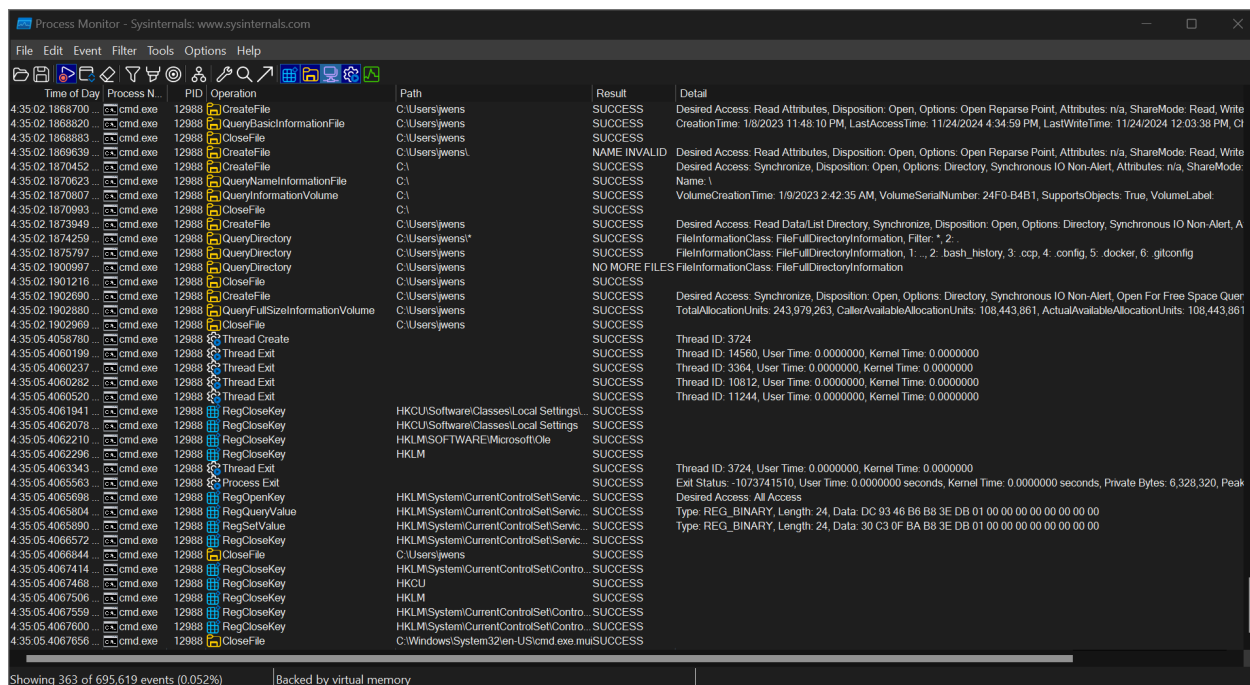
Trace System Calls

Tracing system calls is another way to highlight the differences between Linux and Windows. In Linux, the **strace** command offers a view into the interaction between a process and the kernel. **strace** is invaluable when it comes to debugging a program that is failing to execute properly (GeeksforGeeks, 2024.) Let's take, for example, the simple command **ls**, a simple command to list the contents of a directory, one of the most basic and commonly used commands. When we run the command **strace -e open,read,write,close ls** we can see how the **ls** command interacts with the operating system at a very low level. The **strace** utility reveals that **ls** first opens the current directory with some system read calls made by the **ls** process. These calls involve reading 832 bytes of data at a time from the file descriptor 3, which is associated with the open directory. The hex displayed represents the binary data processed by **ls**. The next line shows a single system write call that outputs the processed directory content to the terminal to show the directory names Desktop, Documents, Downloads, etc. This got written to file descriptor 1 which corresponds to standard output to the terminal. We can see that 72 bytes of data were written. The exit status **+++ exited with 0 +++** confirms that **ls** executed successfully, as 0 is the code for a successful operation.

```
jwensink@vbox:~$ strace -e open,read,write ls
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\260\203\0\0\0\0\0\0"..., 832) = 832
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\4\0\0\0\0\0\0"..., 832) = 832
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
write(1, "Desktop Documents Downloads M"...72Desktop Documents Downloads Music Pictures      Public Tem
ates Videos
) = 72
+++ exited with 0 +++
```

In Windows, we will need to use a system tool called Process Monitor to capture and analyze a process's system interactions. Process Monitor is another part of the Sysinternals suite.

Similar to Process Explorer, ProcMon provides a real-time view of all file systems, registry, and process/thread activity that occurs in the system. For a demonstration, I used Windows Command Prompt to list the contents of a directory using the command *dir*. With ProcMon, we can trace the operations performed by cmd.exe, such as file access, directory queries, and cleanup. The captured events can provide insights into how the *dir* command interacts with the system at a low level. This can be a powerful tool for debugging and understanding process/thread behaviors in the Windows OS.



CPU Affinity

Setting CPU affinity allows a process to run on specific CPUs, which can be useful for optimizing a process's performance. In Linux, *taskset* is a command that can be used to set the CPU affinity of a process (Gökmen, 2022.) To get started with a demonstration, we will need to create a long-running process, *sleep* seems like a good choice. To do this, I opened a terminal

Process Management Utilities

10

and used the command ***sleep 1000 &*** to start a background process that will run for 1000 seconds.

The ***&*** modifier allows the process to run in the background and will keep the terminal free for further commands. The terminal spits out the PID (3978), and we can confirm that the process is active using the command ***ps -p 3978***.

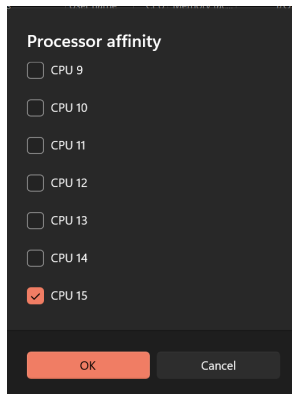
```
jwensink@vbox:~$ sleep 1000 &
[1] 3978
jwensink@vbox:~$ ps -p 3978
  PID TTY          TIME CMD
 3978 pts/0    00:00:00 sleep
jwensink@vbox:~$
```

Now we're ready to run ***taskset -p 3978*** to show the CPUs available. The output ***ff*** means that the process currently has access to all eight available CPUs, as each bit in the mask will correspond to a CPU core. In hex, ***ff*** corresponds to 11111111, meaning that the process can run on all eight allocated CPUs.

When we execute the command ***taskset -p 1 3978***, the terminal outputs to confirm that the PID's new affinity mask is ***1***. We confirm that the ***sleep*** process is now limited to running on CPU 0 as the output to ***taskset -p 3978*** is ***pid 3978's current affinity mask: 1***. Since this is just a demonstration, we will kill the process using the command ***kill 3978***. Pretty cool.

```
jwensink@vbox:~$ sleep 1000 &
[1] 3978
jwensink@vbox:~$ ps -p 3978
  PID TTY          TIME CMD
 3978 pts/0    00:00:00 sleep
jwensink@vbox:~$ taskset -p 3978
pid 3978's current affinity mask: ff
jwensink@vbox:~$ taskset -p 1 3978
pid 3978's current affinity mask: ff
pid 3978's new affinity mask: 1
jwensink@vbox:~$ taskset -p 3978
pid 3978's current affinity mask: 1
jwensink@vbox:~$ kill 3978
[1]+  Terminated                  sleep 1000
jwensink@vbox:~$
```

In Windows the process is much more straightforward. We start by running the command prompt and open Windows Task Manager. In the details tab, I located cmd.exe and simply right click to set affinity. A nice graphical display appears showing CPUs 0-15 with checkboxes that can be selected or unselected, I selected only CPU 15 for this demonstration.



Zombie Processes

A zombie process occurs when a child process has terminated, but its parent has not yet read its exit status, leaving an entry in the process table. Besides having a cool name, it can be important to understand the implications of zombies not because they are consuming system resources but because they can still indicate a problem with parent-child process handling. Once again, we head to the terminal and start a script to generate a zombie process. I asked ChatGPT (OpenAI, 2024) how to demonstrate this functionality, and it suggested a bash script to initiate a child process using *sleep*, which runs in the background while the parent process quickly exits without collecting the exit status of the child. This caused the child process to remain in a terminated state but still present in the process table, becoming a zombie. To monitor this

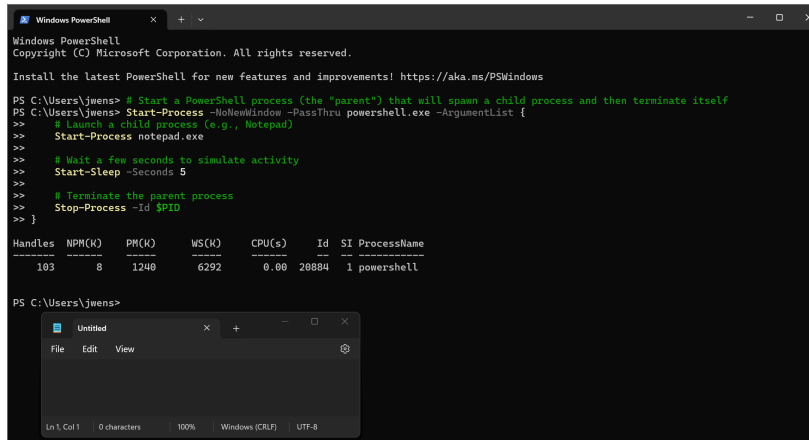
process, we can use the *ps* command with *grep Z* to identify any processes that are flagged as zombies in the status column.

```
jwensink@vbox:~$ bash -c 'sleep 100 & exec sleep 0'
jwensink@vbox:~$ ps aux | grep Z
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
jwensink  5812  0.0  0.0 230336  2284 pts/0    S+   17:23   0:00 grep --color=auto Z
jwensink@vbox:~$
```

In Windows, child processes are cleaned up automatically when the parent exits, and Windows won't retain zombie-like processes in the process table as Linux does. Rather, Windows ensures that any resources associated with a terminated process are promptly reclaimed by the operating system. This eliminates the need for a parent process to explicitly 'reap' their child processes. However, an orphaned process can occur if a parent process terminates abruptly without properly shutting down its children. In such cases, the child will continue to run independently, consuming system resources until they are manually terminated or cleaned up by the system. Again, I asked ChatGPT (2024) how I would be able to demonstrate this, and it recommended a PowerShell script to simulate a parent process of creating and abandoning a child process. Here we can clearly see that notepad.exe is still running even though its parent process has terminated.

Process Management Utilities

13



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\jwens> # Start a PowerShell process (the "parent") that will spawn a child process and then terminate itself
PS C:\Users\jwens> Start-Process -NoNewWindow -PassThru powershell.exe -ArgumentList {
>> # Launch a child process (e.g., Notepad)
>> Start-Process notepad.exe
>>
>> # Wait a few seconds to simulate activity
>> Start-Sleep -Seconds 5
>>
>> # Terminate the parent process
>> Stop-Process -Id $PID
>> }
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
103	8	1240	6292	0.00	20884	1	powershell

PS C:\Users\jwens>

Conclusion

Linux and Windows clearly have different approaches to process/thread management due to their differing design philosophies and underlying architecture. Linux seems to provide a much more granular control over processes through the command line interface, whereas Windows seems to provide a much more graphical delivery of processes through a variety of system tools. Windows Task Manager is a tremendously powerful tool that is easy to use and familiar to the general public, however, other tools like Process Monitor and Process Explorer can provide even greater insights into what is happening behind the scenes. Ultimately, both systems offer control over user-level and kernel-level processes/threads but require different tools and methods to achieve similar goals.

References

Ahao, N. (2024, July 31). *How to list namespaces in Linux*. Baeldung. Retrieved November 24, 2024, from <https://www.baeldung.com/linux/list-namespaces>

GeeksforGeeks. (2024, September 2). Strace command in Linux with examples. GeeksforGeeks. Retrieved November 24, 2024, from <https://www.geeksforgeeks.org/strace-command-in-linux-with-examples/>

Microsoft. (2023, March 17). *Isolation modes*. Microsoft Learn. Retrieved November 24, 2024, from <https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/hyperv-container>

OpenAI. (2024). *Chat with ChatGPT-4o*. Retrieved November 24, 2024, from <https://chat.openai.com>